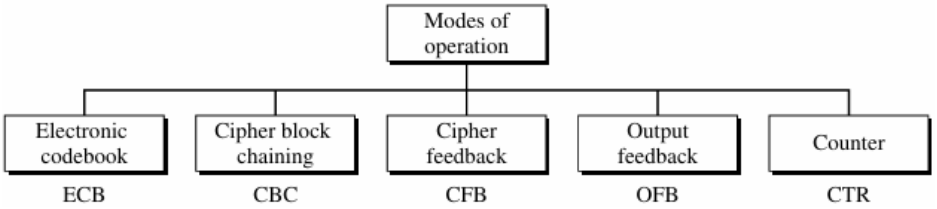
Unit 3

Created	@June 5, 2025 10:20 PM
Class	CNS

Use of Modern Block Ciphers

Modes of operation have been devised to encipher text of any size employing either DES or AES.

Figure 8.1 *Modes of operation*



Electronic code book

The simplest mode of operation is called the electronic codebook (ECB) mode. The plaintext is divided into N blocks. The block size is n bits. If the plaintext size is not a multiple of the block size, the text is padded to make the last block the same size as the other blocks. The same key is used to encrypt and decrypt each block. Figure 8.2 shows the encryption and decryption in this mode.

Example 8.2

This mode is called *electronic codebook* because one can precompile 2^K codebooks (one for each key) in which each codebook has 2^n entries in two columns. Each entry can list the plaintext and

the corresponding ciphertext blocks. However, if K and n are large, the codebook would be far too large to precompile and maintain.

Security Issues:
Following are security issues in ECB mode:

1. Patterns at the block level are preserved.
2. The block independency creates opportunities for exchange some ciphertext blocks without knowing the key.

Error Propagation: A single bit error in transmission can create errors in several (normally half of the bits or all of the bits) in the corresponding block. However, the error does not have any effect on the other blocks.

Ciphertext Stealing

In ECB mode, padding must be added to the last block if it is not n bits long. Padding is not always possible. For example, when the ciphertext needs to be stored in the buffer where the plaintext was previously stored, plaintext and ciphertext must be the same. A technique called **ciphertext stealing (CTS)** can make it possible to use ECB mode without padding. In this technique the last two plaintext blocks, P_{N-1} and P_N, are encrypted differently and out of order, as shown below, assuming that P_{N-1} has n bits and P_N has m bits, where $m \leq n$.

$$X = E_K(P_{N-1}) \quad \rightarrow \quad C_N = head_m(X)$$
$$Y = P_N \mid tail_{n-m}(X) \quad \rightarrow \quad C_{N-1} = E_K(Y)$$

The $head_m$ function selects the leftmost m bits; the $tail_{n-m}$ function selects the rightmost $n - m$ bits. The detailed diagram and the procedure of the encryption and decryption are left as exercises.

Applications

The ECB mode of operation is not recommended for encryption of messages of more than one block to be transferred through an insecure channel. If the message is short enough to fit in one block, the security issues and propagation errors are tolerable.

One area where the independency of the ciphertext block is useful is where records need to be encrypted before they are stored in a database or decrypted before they are retrieved. Because the order of encryption and decryption of blocks is not important in this mode, access to the database can be random if each record is a block or multiple blocks. A record can be retrieved from the middle, decrypted, and encrypted after modification without affecting other records.

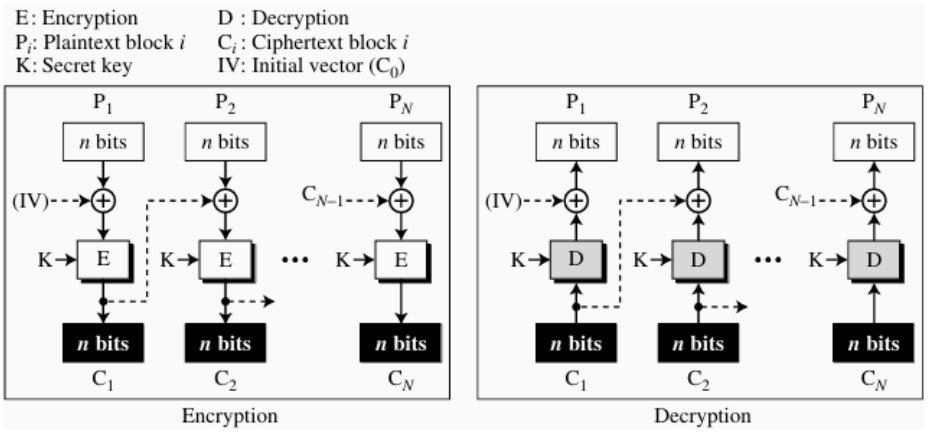
Another advantage of this mode is that we can use parallel processing if we need to create, for example, a very huge encrypted database.

Cipher Block chaining mode

Cipher Block Chaining (CBC) Mode

The next evolution in the operation mode is the **cipher block chaining (CBC) mode.** In CBC mode, each plaintext block is exclusive-ored with the previous ciphertext block before being encrypted. When a block is enciphered, the block is sent, but a copy of it is kept in memory to be used in the encryption of the next block. The reader may wonder about the initial block. There is no ciphertext block before the first block. In this case, a phony block called the **initialization vector (IV)** is used.The sender and receiver agree upon a specific predetermined IV. In other words, an IV is used instead of the nonexistent C_0. Figure 8.3 shows CBC mode. At the sender side, exclusive-oring is done before encryption; at the receiver site, decryption is done before exclusive-oring.

Figure 8.3 *Cipher block chaining (CBC) mode*



Security Issues

Following are two of the security issues in CBC mode:

1. In CBC mode, equal plaintext blocks belonging to the same message are enciphered into different ciphertext blocks. In other words, the patterns at the block

levels are not preserved. However, if two messages are equal, their encipherment is the same if they use the same IV. As a matter of fact, if the first M blocks in two different messages are equal, they are enciphered into equal blocks unless different IVs are used. For this reason, some people recommend the use of a timestamp as an IV.

2. Eve can add some ciphertext blocks to the end of the ciphertext stream.

Error Propagation

In CBC mode, a single bit error in ciphertext block C_j during transmission may create error in most bits in plaintext block P_j during decryption. However, this single error toggles only one bit in plaintext block P_{j+1} (the bit in the same location). The proof of this fact is left as an exercise. Plaintext blocks P_{j+2} to P_N are not affected by this single bit error. A single bit error in ciphertext is *self-recovered*.

Applications

The CBC mode of operation can be used to encipher messages. However, because of chaining mechanism, parallel processing is not possible. CBC mode is not used to

encrypt and decrypt random-access Þles records because encryption and decryption require access to the previous records. As we will see in Chapter 11, CBC mode is also used for authentication.

Cipher Text feedback mode

Cipher Feedback (CFB) Mode

ECB and CBC modes encrypt and decrypt blocks of the message. The block size, predetermined by the underlying cipher; for example, $n = 64$ for DES and $n = 128$ for AES. In some situations, we need to use DES or AES as secure ciphers, but the plaintext or ciphertext block sizes are to be smaller. For example, to encrypt and decrypt ASCII 8-bit characters, you would not want to use one of the traditional ciphers discussed in Chapter 3 because they are insecure. The solution is to use DES or AES in cipher feedback (CFB) mode. In this mode the size of the block used in DES or AES is n, but the size of the plaintext or ciphertext block is r, where $r \leq n$.

The idea is to use DES or AES, not for encrypting the plaintext or decrypting the ciphertext, but to encrypt or decrypt the contents of a shift register, S, of size n. Encryption is done by exclusive-oring an r-bit plaintext block with r bits of the shift register. Decryption is done by exclusive-oring an r-bit ciphertext block with r bits of the shift register. For each block, the shift register, S_i, is made by shifting the shift register, S_{i-1} (previous shift register) bits to the left and filling the rightmost r bits with C_{i-1}. S_i is then encrypted to T_i. Only the rightmost r bits of T_i are exclusive-ored with the plaintext block P_i to make the C_i. Note that S_1, which is not shifted, is set to the IV for the first block.

Figure 8.4 shows the CFB mode for enciphering; deciphering is the same, but the roles of plaintext blocks (P_i) and ciphertext blocks (C_i) are switched. Note that both encipherment and decipherment use the encryption function of the underlying block cipher (DES or AES, for example).

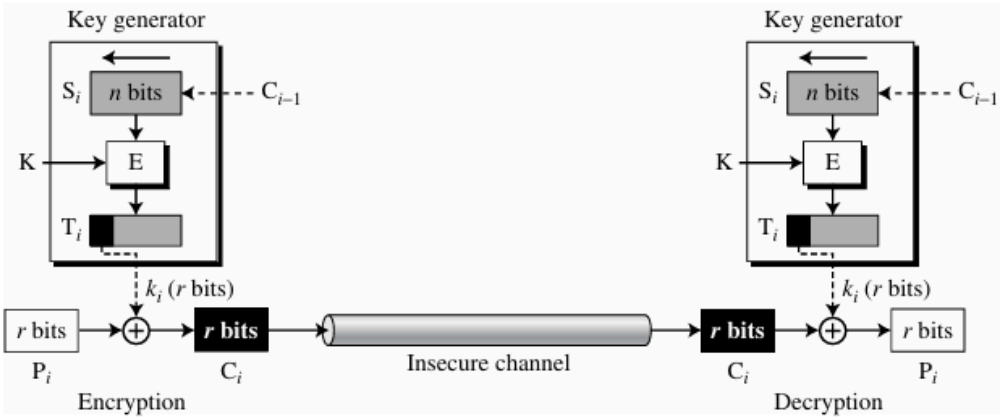
an exercise.

One interesting point about this mode is that no padding is required because the size of the blocks, *r*, is normally chosen to fit the data unit to be encrypted (a character, for example). Another interesting point is that the system does not have to wait until it has received a large block of data (64 bits or 128 bits) before starting the encryption. The encrypting process is done for a small block of data (such as a character). These two advantages come with a disadvantage. CFB is less efficient than CBC or ECB, because it needs to apply the encryption function of underlying block cipher for each small block of size *r*.

CFB as a Stream Cipher

Although CFB is an operation mode for using block ciphers such as DES or AES, the result is a stream cipher. In fact, it is a nonsynchronous stream cipher in which the key stream is dependent on the ciphertext. Figure 8.5 shows the point of the encryption and decryption where the key generator is conspicuous.

Figure 8.5 *Cipher feedback (CFB) mode as a stream cipher*



Security Issues

There are three primary security issues in CFB mode:

1. Just like CBC, the patterns at the block level are not preserved.
2. More than one message can be encrypted with the same key, but the value of the IV should be changed for each message. This means that Alice needs to use a different IV each time she sends a message.
3. Eve can add some ciphertext block to the end of the ciphertext stream.

Error Propagation

In CFB, a single bit error in ciphertext block C_j during transmission creates a single bit error (at the same position) in plaintext block P_j. However, most of the bits in the

following plaintext blocks are in error (with 50 percent probability) as long as some bits of C_j are still in the shift register. The calculation of the number of affected blocks is left as an exercise. After the shift register is totally refreshed, the system recovers from the error.

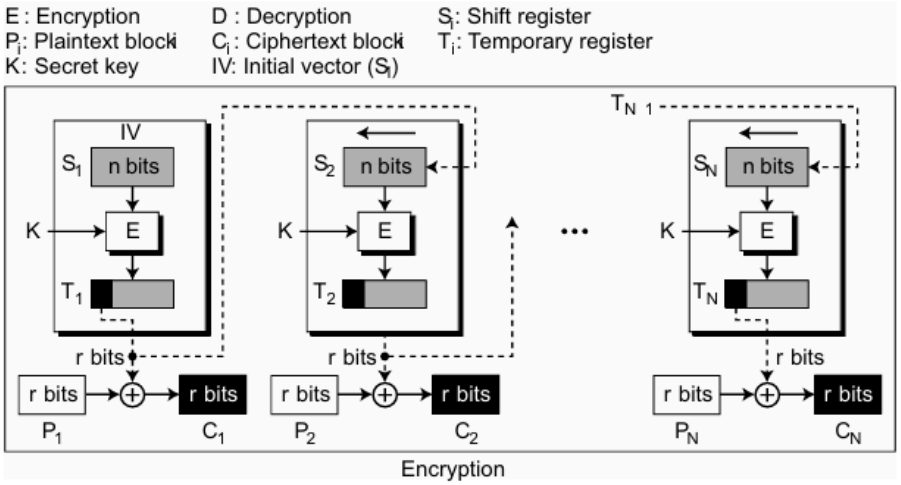
Application
The CFB mode of operation can be used to encipher blocks of small size such as one character or bit at a time. There is no need for padding because the size of the plaintext block is normally fixed (8 for a character or 1 for a bit).

Output Feedback mode

Output Feedback (OFB) Mode

Output feedback (OFB) modeis very similar to CFB mode, with one difference: each bit in the ciphertext is independent of the previous bit or bits. This avoids error propagation. If an error occurs in transmission, it does not affect the bits that follow. Note that, like CFB, both the sender and the receiver use the encryption algorithm. Figure 8.6 shows OFB mode.

Figure 8.6 Encryption in output feedback (OFB) mode



OFB as a Stream Cipher:
OFB, like CFB, creates a stream cipher out of the underlying block cipher. The key
stream, however, is independent from the plaintext or ciphertext, which means that the

Security Issues

Following are two of the security issues in OFB mode:

1. Just like the CFB mode, patterns at the block level are not preserved.
2. Any change in the ciphertext affects the plaintext encrypted at the receiver side.

Error Propagation

A single error in the ciphertext affects only the corresponding bit in the plaintext.

Special Case

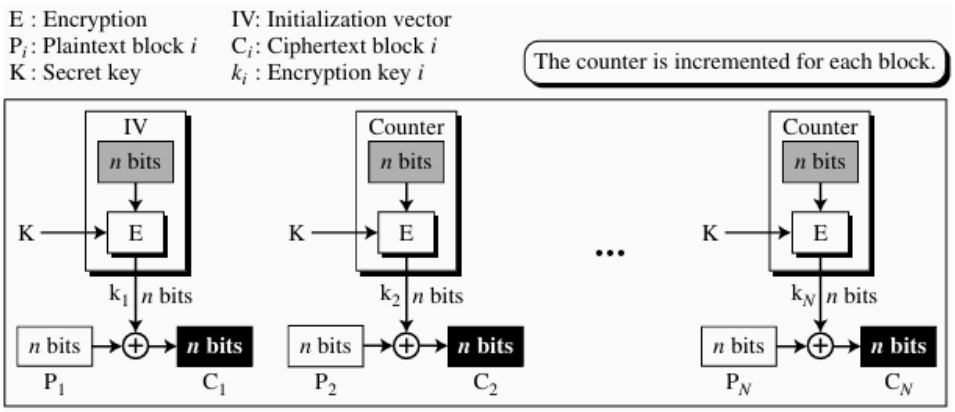
If the blocks in the text and the underlying cipher are of the same size ($n = r$), the encryption/decryption becomes simpler, but we leave the discovery of the diagram and the algorithm as an exercise.

Counter Mode

Counter (CTR) Mode

In the **counter (CTR) mode,** there is no feedback. The pseudorandomness in the key stream is achieved using a counter. An n-bit counter is initialized to a pre-determined value (IV) and incremented based on a predefined rule (mod 2^n). To provide a better randomness, the increment value can depend on the block number to be incremented. The plaintext and ciphertext block have the same block size as the underlying cipher (e.g., DEA or AES). Plaintext blocks of size n are encrypted to create ciphertext blocks of size n. Figure 8.8 shows the counter mode.

Figure 8.8 *Encryption in counter (CTR) mode*



Use Of Stream Ciphers

RC4

RC4 is a byte-oriented stream cipher in which a byte (8 bits) of a plaintext is exclusive-ored with a byte of key to produce a byte of a ciphertext. The secret key,
from which the one-byte keys in the key stream are generated, can contain anywhere
from 1 to 256 bytes.

State

RC4 is based on the concept of a state. At each moment, a state of 256 bytes is active, from which one of the bytes is randomly selected to serve as the key for encryption. The idea can be shown as an array of bytes:

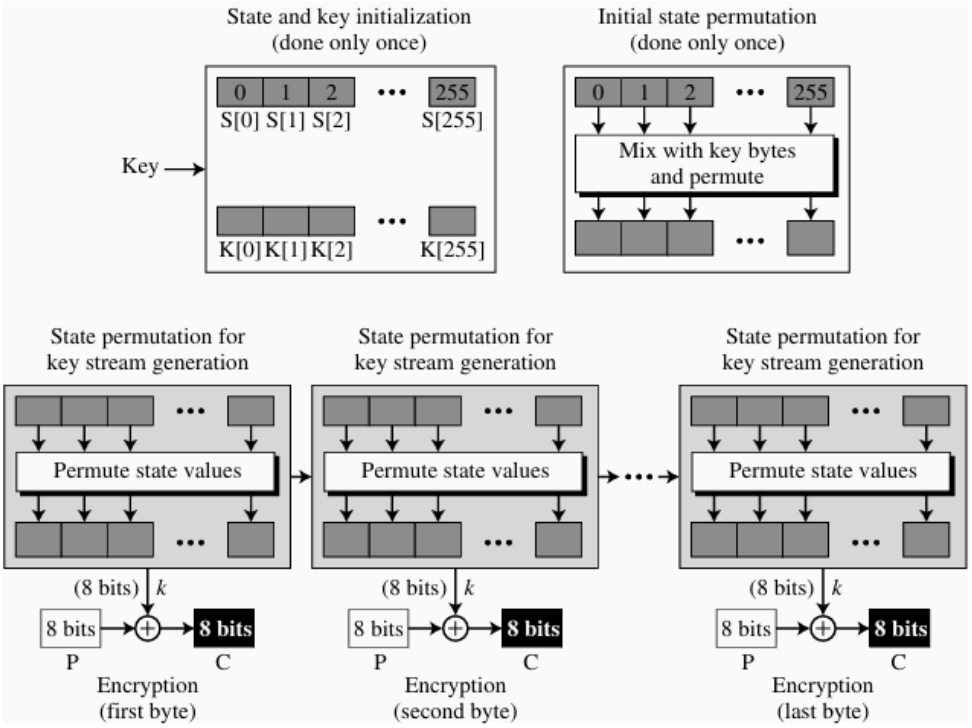
$$S[0] \quad S[1] \quad S[2] \quad \cdots \quad S[255]$$

Note that the indices of the elements range between 0 and 255. The contents of each element is also a byte (8 bits) that can be interpreted as an integer between 0 to 255.

The Idea

Figure 8.10 shows the whole idea of RC4. The first two boxes are performed only once (initializing); the permutation for creating stream key is repeated as long as there are plaintext bytes to encrypt.

Figure 8.10 *The idea of RC4 stream cipher*



Initialization Initialization is done in two steps:

1. In the first step, the state is initialized to values 0, 1, …, 255. A key array, K[0], K[1], …, K[255] is also created. If the secret key has exactly 256 bytes, the bytes are copied to the K array; otherwise, the bytes are repeated until the K array is filled.

```
for (i = 0 to 255)
{
    S[i]  ← i
    K[i]  ← Key [i mod KeyLength]
}
```

2. In the second step, the initialized state goes through a permutation (swapping the elements) based on the value of the bytes in K[*i*]. The key byte is used only in this step to define which elements are to be swapped. After this step, the state bytes are completely shuffled.

```
j ← 0
for (i = 0 to 255)
{
    j ← (j + S[i] + K[i]) mod 256
    swap (S[i] , S[j])
}
```

Key Stream Generation The keys in the key stream, the *k*'s, are generated, one by one. First, the state is permuted based on the values of state elements and the values of two individual variables, *i* and *j*. Second, the values of two state elements in positions *i* and *j* are used to define the index of the state element that serves as *k*. The following code is repeated for each byte of the plaintext to create a new key element in the key stream. The variables *i* and *j* are initialized to 0 before the first iteration, but the values are copied from one iteration to the next.

```
i  ← (i + 1) mod 256
j  ← (j + S[i]) mod 256
swap (S [i] , S[j])
k  ← S [(S[i] + S[j]) mod 256]
```

Encryption or Decryption After *k* has been created, the plaintext byte is encrypted with *k* to create the ciphertext byte. Decryption is the reverse process.

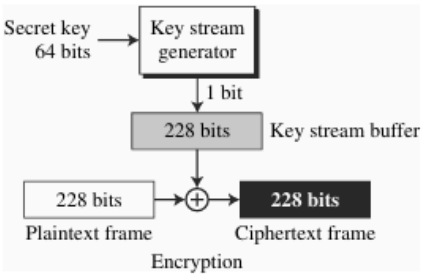
Security Issues

It is believed that the cipher is secure if the key size is at least 128 bits (16 bytes). There are some reported attacks for smaller key sizes (less than 5 bytes), but the protocols that use RC4 today all use key sizes that make RC4 secure. However, like many other ciphers, it is recommended the different keys be used for different sessions. This prevents Eve from using differential cryptanalysis on the cipher.

A5/1

In this section we introduce a stream cipher that uses LFSRs (see Chapter 5) to create a bit stream: **A5/1.** A5/1 (a member of the A5 family of ciphers) is used in the **Global System for Mobile Communication (GSM),** a network for mobile telephone communication. Phone communication in GSM is done as a sequence of 228-bit frames in which each frame lasts 4.6 milliseconds. A5/1 creates a bit stream out of a 64-bit key. The bit streams are collected in a 228-bit buffer to be exclusive-ored with a 228-bit frame, as shown in Figure 8.11.

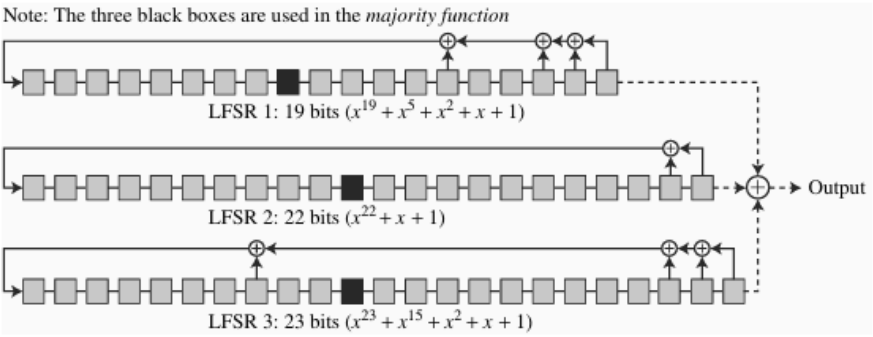
Figure 8.11 *General outline of A5/1*



Key Generator

A5/1 uses three LFSRs with 19, 22, and 23 bits. The LFSRs, the characteristic polynomials, and the clocking bits are shown in Figure 8.12.

Figure 8.12 *Three LFSR's in A5/1*



An **invertible function** is a function that associates each element in the range with exactly one element in the domain.

One-Way Function

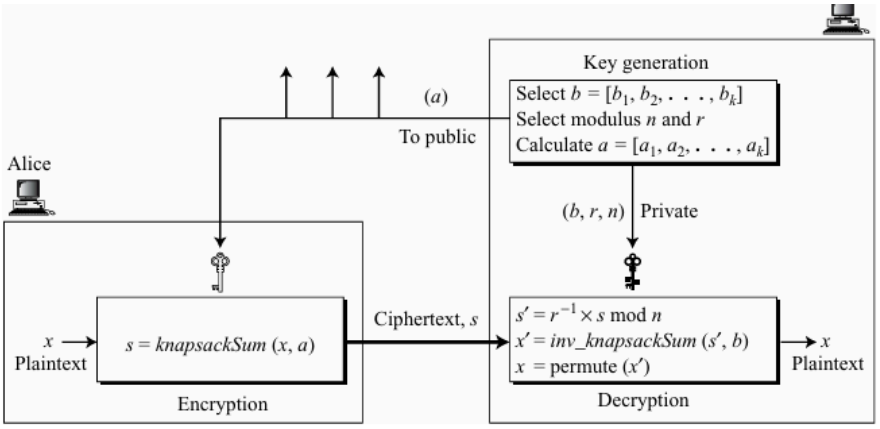
A **one-way function (OWF)** is a function that satisfies the following two properties:

1. f is easy to compute. In other words, given x, $y = f(x)$ can be easily computed.
2. f^{-1} is difficult to compute. In other words, given y, it is computationally infeasible to calculate $x = f^{-1}(y)$.

Trapdoor One-Way Function

A **trapdoor one-way function (TOWF)** is a one-way function with a third property:

3. Given y and a **trapdoor** (secret), x can be computed easily.



Key Generation

a. Create a superincreasing k-tuple $b = [b_1, b_2, \ldots, b_k]$
b. Choose a modulus n, such that $n > b_1 + b_2 + \cdots + b_k$

c. Select a random integer r that is relatively prime with n and $1 \le r \le n - 1$.
d. Create a temporary k-tuple $t = [t_1, t_2, \ldots, t_k]$ in which $t_i = r \times b_i \bmod n$.
e. Select a permutation of k objects and find a new tuple $a = permute\ (t)$.
f. The public key is the k-tuple a. The private key is n, r, and the k-tuple b.

the ciphertext.

Decryption

Bob receives the ciphertext s.

a. Bob calculates $s' = r^{-1} \times s \bmod n$.
b. Bob uses *inv_knapsackSum* to create x'.
c. Bob permutes x' to find x. The tuple x is the recovered plaintext.

Example 10.4

This is a trivial (very insecure) example just to show the procedure.

1. Key generation:
   a. Bob creates the superincreasing tuple $b = [7, 11, 19, 39, 79, 157, 313]$.
   b. Bob chooses the modulus $n = 900$ and $r = 37$, and [4 2 5 3 1 7 6] as permutation table.
   c. Bob now calculates the tuple $t = [259, 407, 703, 543, 223, 409, 781]$.
   d. Bob calculates the tuple a = permute $(t) = [543, 407, 223, 703, 259, 781, 409]$.
   e. Bob publicly announces a; he keeps n, r, and b secret.
2. Suppose Alice wants to send a single character "g" to Bob.
   a. She uses the 7-bit ASCII representation of "g", $(1100111)_2$, and creates the tuple $x = [1, 1, 0, 0, 1, 1, 1]$. This is the plaintext.
   b. Alice calculates $s = knapsackSum\ (a, x) = 2165$. This is the ciphertext sent to Bob.
3. Bob can decrypt the ciphertext, $s = 2165$.
   a. Bob calculates $s' = s \times r^{-1} \bmod n = 2165 \times 37^{-1} \bmod 900 = 527$.
   b. Bob calculates $x' = Inv_knapsackSum\ (s', b) = [1, 1, 0, 1, 0, 1, 1]$.
   c. Bob calculates $x = permute\ (x') = [1, 1, 0, 0, 1, 1, 1]$. He interprets the string $(1100111)_2$ as the character "g".

Introduction

RSA uses two exponents, e and d, where e is public and d is private. Suppose P is the plaintext and C is the ciphertext. Alice uses $C = P^e \bmod n$ to create ciphertext C from plaintext P; Bob uses $P = C^d \bmod n$ to retrieve the plaintext sent by Alice. The modulus n, a very large number, is created during the key generation process, as we will discuss later.

Two Algebraic Structures

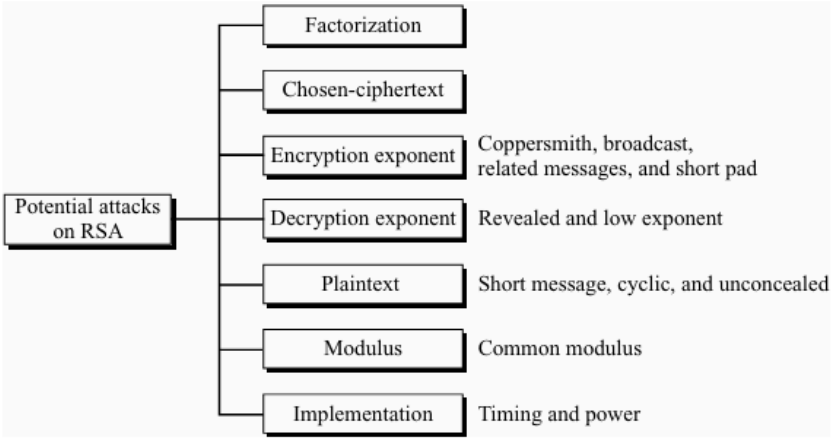
RSA uses two algebraic structures: a ring and a group.

Encryption/Decryption Ring Encryption and decryption are done using the commutative ring $\mathbf{R} = <\mathbf{Z}_n, +, \times>$ with two arithmetic operations: addition and multiplication. In RSA, this ring is public because the modulus n is public. Anyone can send a message to Bob using this ring to do encryption.

Key-Generation Group RSA uses a multiplicative group $\mathbf{G} = <\mathbf{Z}_{\phi(n)}^*, \times>$ for key generation. This group supports only multiplication and division (using multiplicative inverses), which are needed for generating public and private keys. This group is hidden from the public because its modulus, $\phi(n)$, is hidden from the public. We will see shortly that if Eve can find this modulus, she can easily attack the cryptosystem.

RSA uses two algebraic structures:
a public ring $\mathbf{R} = <\mathbf{Z}_n, +, \times>$ and a private group $\mathbf{G} = <\mathbf{Z}_{\phi(n)}^*, \times>$.

Figure 10.8 *Taxonomy of potential attacks on RSA*



10.3 RABIN CRYPTOSYSTEM

The **Rabin cryptosystem,** devised by M. Rabin, is a variation of the RSA cryptosystem. RSA is based on the exponentiation congruence; Rabin is based on quadratic congruence. The Rabin cryptosystem can be thought of as an RSA cryptosystem in which the value of e and d are fixed; $e = 2$ and $d = 1/2$. In other words, the encryption is $C \equiv P^2 \pmod{n}$ and the decryption is $P \equiv C^{1/2} \pmod{n}$.

The public key in the Rabin cryptosystem is n; the private key is the tuple (p, q). Everyone can encrypt a message using n; only Bob can decrypt the message using p and q. Decryption of the message is infeasible for Eve because she does not know the values of p and q. Figure 10.10 shows the encryption and decryption.

example, assume the following scenario. Alice wants to send the same message to three recipients with the same public exponent $e = 3$ and the moduli n_1, n_2, and n_3.

$$C_1 = P^3 \bmod n_1 \qquad C_2 = P^3 \bmod n_2 \qquad C_3 = P^3 \bmod n_3$$

Applying the Chinese remainder theorem to these three equations, Eve can find an equation of the form $C' = P^3 \bmod n_1 n_2 n_3$. This means that $P^3 < n_1 n_2 n_3$. This means $C' = P^3$ is in regular arithmetic (not modular arithmetic). Eve can find the value of $C' = P^{1/3}$.

Related Message Attack The **related message attack,** discovered by Franklin Reiter, can be briefly described as follows. Alice encrypts two plaintexts, P_1 and P_2, and encrypts them with $e = 3$ and sends C_1 and C_2 to Bob. If P_1 is related to P_2 by a linear function, then Eve can recover P_1 and P_2 in a feasible computation time.

Short Pad Attack The **short pad attack,** discovered by Coppersmith, can be briefly described as follows. Alice has a message M to send to Bob. She pads the message with r_1, encrypts the result to get C_1, and sends C_1 to Bob. Eve intercepts C_1 and drops it. Bob informs Alice that he has not received the message, so Alice pads the message again with r_2, encrypts it, and sends it to Bob. Eve also intercepts this message. Eve now has C_1 and C_2, and she knows that they both are ciphertexts belonging to the same plaintext. Coppersmith proved that if r_1 and r_2 are short, Eve may be able to recover the original message M.

Attacks on the Decryption Exponent

Two forms of attacks can be launched on the decryption exponent: **revealed decryption exponent attack** and **low decryption exponent attack.** They are discussed briefly.

Revealed Decryption Exponent Attack It is obvious that if Eve can find the decryption exponent, d, she can decrypt the current encrypted message. However, the attack does not stop here. If Eve knows the value of d, she can use a probabilistic algorithm (not discussed here) to factor n and find the value of p and q. Consequently, if Bob changes only the compromised decryption exponent but keeps the same modulus, n, Eve will be able to decrypt future messages because she has the factorization of n. This means that if Bob finds out that the decryption exponent is compromised, he needs to choose new value for p and q, calculate n, and create totally new private and public keys.

In RSA, if d is comprised, then p, q, n, e, and d must be regenerated.

Low Decryption Exponent Attack Bob may think that using a small private-key d, would make the decryption process faster for him. Wiener showed that if $d < 1/3\ n^{1/4}$, a special type of attack based on *continuous fraction,* a topic discussed in number theory, can jeopardize the security of RSA. For this to happen, it must be the case that $q < p < 2q$. If these two conditions exist, Eve can factor n in polynomial time.

In RSA, the recommendation is to have $d \geq 1/3\ n^{1/4}$ to prevent low decryption exponent attack.

Plaintext Attacks

Plaintext and ciphertext in RSA are permutations of each other because they are integers in the same interval (0 to $n - 1$). In other words, Eve already knows something about the plaintext. This characteristic may allow some attacks on the plaintext. Three

Plaintext Attacks

Plaintext and ciphertext in RSA are permutations of each other because they are integers in the same interval (0 to $n - 1$). In other words, Eve already knows something about the plaintext. This characteristic may allow some attacks on the plaintext. Three attacks have been mentioned in the literature: short message attack, cycling attack, and unconcealed attack.

Short Message Attack In the **short message attack,** if Eve knows the set of possible plaintexts, she then knows one more piece of information in addition to the fact that the ciphertext is the permutation of plaintext. Eve can encrypt all of the possible messages until the result is the same as the ciphertext intercepted. For example, if it is known that Alice is sending a four-digit number to Bob, Eve can easily try plaintext numbers from 0000 to 9999 to find the plaintext. For this reason, short messages must be padded with random bits at the front and the end to thwart this type of attack. It is strongly recommended that messages be padded with random bits before encryption using a method called OAEP, which is discussed later in this chapter.

Cycling Attack The **cycling attack** is based on the fact that if the ciphertext is a permutation of the plaintext, the continuous encryption of the ciphertext will eventually result in the plaintext. In other words, if Eve continuously encrypts the intercepted ciphertext C, she will eventually get the plaintext. However, Eve does not know what the plaintext is, so she does not know when to stop. She needs to go one step further. When she gets the ciphertext C again, she goes back one step to find the plaintext.

Algorithm 10.2 *RSA Key Generation*

```
RSA_Key_Generation
{
    Select two large primes p and q such that p ≠ q.
    n ← p × q
    ϕ(n) ← (p − 1) × (q − 1)
    Select e such that 1 < e < ϕ(n) and e is coprime to ϕ(n)
    d ← e^−1 mod ϕ(n)                    // d is inverse of e modulo ϕ(n)
    Public_key ← (e, n)                  // To be announced publicly
    Private_key ← d                      // To be kept secret
    return Public_key and Private_key
}
```

In RSA, the tuple (e, n) is the public key; the integer d is the private key.

Procedure

Key generation, encryption, and decryption are described below.

Key Generation

Bob uses the steps shown in Algorithm 10.6 to create his public key and private key.

Algorithm 10.6 *Key generation for Rabin cryptosystem*

```
Rabin_Key_Generation
{
    Choose two large primes p and q in the form 4k + 3 and p ≠ q.
    n ← p × q
    Public_key ← n                          // To be announced publicly
    Private_key ← (p, q)                    // To be kept secret
    return Public_key and Private_key
}
```

```
Rabin_Encryption (n, P)          // n is the public key; P is the ciphertext from Z_n*
{
    C ← P^2 mod n                // C is the ciphertext
    return C
}
```

Although the plaintext P can be chosen from the set $\mathbf{Z}_n$, we have defined the set to be in $\mathbf{Z}_n^*$ to make the decryption easier.

Encryption in the Rabin cryptosystem is very simple. The operation needs only one multiplication, which can be done quickly. This is beneficial when resources are limited. For example, smart cards have limited memory and need to use short CPU time.

Decryption

Bob can use Algorithm 10.8 to decrypt the received ciphertext.

Algorithm 10.8 *Decryption in Rabin cryptosystem*

```
Rabin_Decryption (p, q, C)       // C is the ciphertext; p and q are private keys
{
    a1 ← +(C^((p+1)/4)) mod p
    a2 ← −(C^((p+1)/4)) mod p
    b1 ← +(C^((q+1)/4)) mod q
    b2 ← −(C^((q+1)/4)) mod q
    // The algorithm for the Chinese remainder theorem is called four times.
    P1 ← Chinese_Remainder (a1, b1, p, q)
    P2 ← Chinese_Remainder (a1, b2, p, q)
    P3 ← Chinese_Remainder (a2, b1, p, q)
    P4 ← Chinese_Remainder (a2, b2, p, q)
    return P1, P2, P3, and P4
```

the right answer.

The Rabin cryptosystem is not deterministic: Decryption creates four equally probable plaintexts.

Key Generation

Bob uses the steps shown in Algorithm 10.9 to create his public and private keys.

Algorithm 10.9 *ElGamal key generation*

```
ElGamal_Key_Generation
{
    Select a large prime p
    Select d to be a member of the group G = < Z_p*, × > such that 1 ≤ d ≤ p − 2
    Select e_1 to be a primitive root in the group G = < Z_p*, × >
    e_2 ← e_1^d mod p
    Public_key ← (e_1, e_2, p)                    // To be announced publicly
    Private_key ← d                               // To be kept secret
    return Public_key and Private_key
}
```

Encryption

Anyone can send a message to Bob using his public key. The encryption process is shown in Algorithm 10.10. If the fast exponential algorithm (see Chapter 9) is used, encryption in the ElGamal cryptosystem can also be done in polynomial time complexity.

Algorithm 10.10 *ElGamal encryption*

```
ElGamal_Encryption (e_1, e_2, p, P)               // P is the plaintext
{
    Select a random integer r in the group G = < Z_p*, × >
    C_1 ← e_1^r mod p
    C_2 ← (P × e_2^r) mod p                       // C_1 and C_2 are the ciphertexts
    return C_1 and C_2
}
```

Decryption

Bob can use Algorithm 10.11 to decrypt the ciphertext message received.

Algorithm 10.11 *ElGamal decryption*

```
ElGamal_Decryption (d, p, C_1, C_2)               // C_1 and C_2 are the ciphertexts
{
    P ← [C_2 (C_1^d)^−1] mod p                    // P is the plaintext
    return P
}
```

The bit-operation complexity of encryption or decryption in ElGamal cryptosystem is polynomial.